

게임 서버 프로그래밍 텀프로젝트 보고서

Aetheria Online — 균열의 대륙

High-Performance MMORPG Server

플랫폼 Win32 IOCP · C++20 · x64 Release

월드 규모 2,000 × 2,000 타일 · 단일 심리스 월드

NPC 수 200,000 마리 (4 대 바이옴 + 보스 4 종)

목표 동시접속 5,000 CCU · 무랙(無 lag) 처리

측정 결과 localhost 3,462 CCU @ 0~1ms

2026 년 6 월

목차

1. 프로젝트 개요
2. 개발 환경 및 프로젝트 구성
3. 전체 서버 아키텍처
4. 프로토콜 설계
5. 자료구조
6. 게임 흐름
7. 핵심 알고리즘
8. 게임 콘텐츠 구현
9. 데이터베이스 연동
10. 스크립트(Lua) 연동
11. 성능 최적화 및 동접 측정
12. 채점 기준 대응 정리
13. 한계 및 향후 과제

부록 A. 주요 상수

부록 B. 빌드 및 실행

1. 프로젝트 개요

「Aetheria Online: 균열의 대륙」은 2000 × 2000 타일의 단일 심리스 월드에서 동작하는 2D 탑다운 시야형 MMORPG 이다. 차원의 균열로부터 쏟아져 나온 20 만 마리의 몬스터에 맞서 플레이어(수호자)가 4 대 바이옴을 정화하는 컨셉이며, 본 프로젝트의 핵심 목표는 Win32 IOCP 기반 고성능 서버로 단일 머신에서 5,000 명 이상의 동시 접속을 무랙(無 lag)으로 처리하는 것이다.

1.1 구현 범위 요약

분류	구현 항목
필수 (구현 50%)	이동·충돌, 정적 장애물, NPC AI(Peace/Agro), NPC 이동, 4 방향 전투, DB 영속화, Lua 스크립트, 채팅, 전투 메시지
성능 (20%)	IOCP, 섹터 기반 AOI, Lazy AI, 샤딩 타이머/송신풀, 세션별 송신 코일레싱, Object Pool
게임성 (40%)	4 대 바이옴, 스킬 3 종 + MP, 파티(경험치 공유), 아이템·인벤토리·장비, 연쇄 퀘스트, 보스(2 페이지 패턴 AI)
스크립트 가산점	Lua 기반 NPC 스폰 배치 + 보스 패턴 AI

※ 표절 방지(-20% 회피): 실습 코드를 일절 사용하지 않고 IOCP 래퍼, 패킷 조립(Length-Prefix + 자체 RingBuffer), 타이머(Min-Heap 우선순위 큐), DB 비동기 Job Queue, Lua 바인딩을 모두 처음부터 자체 구현하였다. 제공된 protocol_2026.h 만 기준 프로토콜로 사용하고 그 위에 패킷을 추가 정의했다.

2. 개발 환경 및 프로젝트 구성

2.1 개발 환경

항목	사양
OS	Windows 11
IDE / 빌드	Visual Studio 2022, C++20, x64 Release /O2
네트워크	Win32 IOCP + Overlapped I/O

스크립트	Lua 5.5 (C API 직접 바인딩)
DB	MySQL 8.0 + Connector/ODBC (JSON 파일 풀백)
동시성 라이브러리	Intel TBB concurrent_hash_map
클라이언트	SFML 2.5.1 (시연/검증용 GUI 클라이언트)

2.2 디렉터리 구성

```

2026GameServerTermproject/
├─ MMOSERVER_Termproject/           # 서버 (Visual Studio 솔루션)
│  └─ MMOSERVER_Termproject/
│     ├─ MMOSERVER_Termproject.cpp # IOCP 루프 + 게임 로직 (≈3,400 LOC)
│     ├─ protocol_2026.h           # 제공 프로토콜 + 추가 패킷 정의
│     └─ Core/
│        ├─ Entity.h / NPC.h       # 엔티티·NPC 데이터 모델
│        ├─ World.{h,cpp}          # NPC 스토리지
│        ├─ NpcSpawner.{h,cpp}     # 스폰 스크립트 파서
│        ├─ Map.{h,cpp}            # 장애물 비트맵 + 충돌 판정
│        ├─ AStar.{h,cpp}          # A* 길찾기
│        ├─ LuaVM.{h,cpp}          # Lua 인터프리터 래퍼
│        ├─ TimerManager.{h,cpp}   # 샤딩 Min-Heap 타이머
│        ├─ ObjectPool.h           # free-list 오브젝트 풀
│        ├─ Item.{h,cpp}           # 아이템 카탈로그·드롭
│        ├─ Quest.{h,cpp}          # 퀘스트 카탈로그
│        ├─ GameConfig.h           # 게임 규칙 상수
│        └─ Db/                   # DB 추상화 계층
│           ├─ IDbBackend.h
│           ├─ DbWorker.{h,cpp}    # 비동기 DB 스레드
│           ├─ MySQLOdbcBackend.*
│           └─ JsonFileBackend.*
├─ CLIENT/client_sample/client.cpp  # SFML GUI 클라이언트
└─ data/                          # 런타임 스크립트·설정
   ├─ npc_spawn.txt               # 몬스터 배치 스크립트
   ├─ npc_ai.lua                  # NPC/보스 AI 스크립트
   ├─ obstacles.txt               # 장애물 배치 스크립트
   ├─ items.txt                   # 아이템 카탈로그
   ├─ quests.txt                  # 퀘스트 카탈로그
   ├─ db.ini                      # DB 백엔드 설정
   └─ schema_mysql.sql# MySQL 스키마

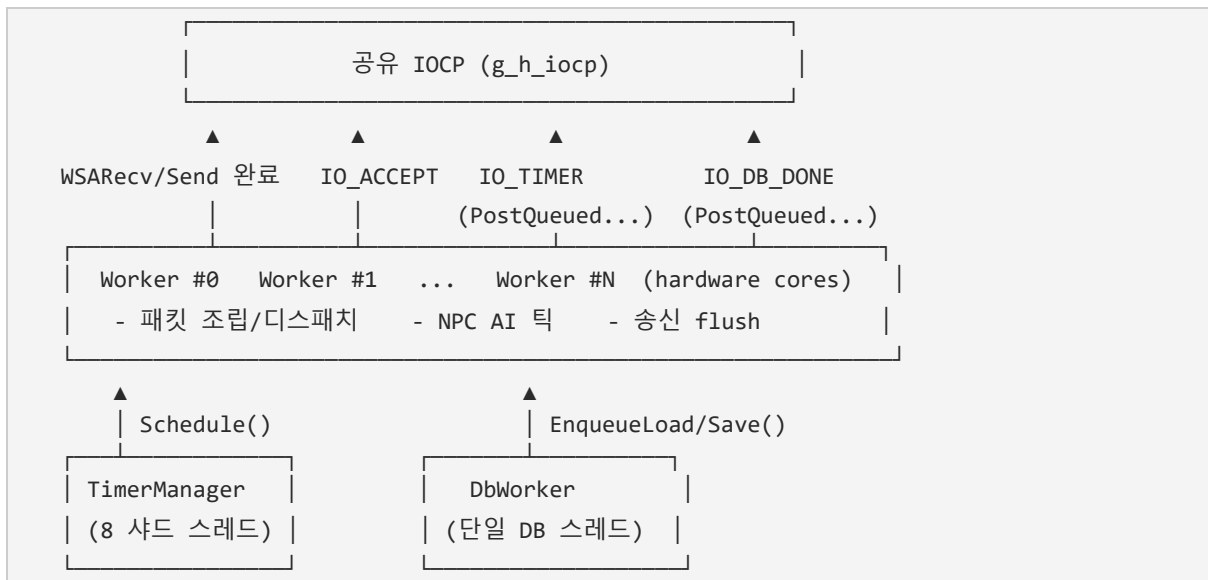
```

게임 규칙(상수)은 `GameConfig.h` 로 분리하여 제공 프로토콜 헤더(`protocol_2026.h`)를 오염시키지 않도록 했다. 데이터 파일은 서버를 어느 디렉터리에서 실행하든 찾을 수 있도록 `data/`, `.././data/`, `../.././data/` 3 단계 후보 경로를 순차 탐색한다.

3. 전체 서버 아키텍처

3.1 스레드 모델

서버는 하나의 공유 IOCP 에 모든 이벤트(네트워크 I/O, 타이머 만기, DB 완료)를 집중시키고, `std::thread::hardware_concurrency()`개의 워커 스레드가 동일한 루프(`worker_thread`)를 돌며 이벤트를 꺼내 처리하는 구조다. 별도의 게임 로직 전용 스레드가 없으며, 게임 로직은 I/O 이벤트를 처리하는 워커 스레드 위에서 그대로 실행된다(I/O + 로직 통합 모델).



- 타이머 만기와 DB 완료는 각각 별도 스레드(TimerManager 샤드, DbWorker)에서 발생하지만, 처리는 `PostQueuedCompletionStatus` 로 공유 IOCP 에 이벤트를 post 하여 워커 스레드가 일관되게 소비한다. 즉 모든 게임 상태 변경은 워커 스레드 컨텍스트에서 일어난다.
- 이 설계 덕분에 타이머·DB 콜백과 네트워크 핸들러가 동일한 동기화 규약(섹터 락, `view_lock`, `atomic`)을 공유한다.

3.2 이벤트 타입 디스패치

`OVERLAPPED_EX` / `TimerOverlapped` / `DbOverlapped` 는 모두 첫 두 필드를 `WSAOVERLAPPED` + `IO_TYPE` 로 동일 레이아웃으로 시작하도록 설계했다. 워커는 완료 통지에서 `IO_TYPE` 만 먼저 읽어 어떤 종류의 이벤트인지 안전하게 분기한다.

IO_TYPE	의미	처리
IO_ACCEPT	신규 연결	세션 생성, IOCP 등록, AcceptEx 재장전
IO_RECV	수신 완료	RingBuffer 적재 → 패킷 조립 → process_packet
IO_SEND	송신 완료	부분전송 되돌림 + 다음 배치 flush
IO_TIMER	타이머 만기	NpcMove/Respawn/HpRegen/MpRegen/AutoSave/ItemExpire
IO_DB_DONE	DB 작업 완료	Load → 캐릭터 스폰 / Save → 결과 로그

4. 프로토콜 설계

4.1 패킷 공통 규약

- Length-Prefix 방식: 모든 패킷의 첫 바이트는 unsigned char size(패킷 전체 바이트 수, 최대 255), 두 번째 필드는 PACKET_TYPE type 이다.
- #pragma pack(push, 1)으로 구조체 패딩을 제거하여 네트워크 바이트 레이아웃을 고정했다.
- 수신 측은 첫 1 바이트(size)만 먼저 엿보고(Peek) 버퍼에 그 크기만큼 데이터가 모였을 때에만 패킷을 추출하므로 TCP 스트림의 분할·몽침을 완전히 처리한다.

4.2 패킷 목록

제공된 protocol_2026.h 의 기본 패킷(LOGIN, MOVE, CHAT, ATTACK, ADD/REMOVE/MOVE_OBJECT, STATUS_CHANGE 등)에 더해 다음 패킷을 추가 정의하였다.

■ 클라이언트 → 서버 (C2S)

패킷	용도
C2S_LOGIN	ID 기반 로그인(없으면 신규 생성)
C2S_MOVE / C2S_TELEPORT	이동 / 텔레포트(스트레스 테스트용)
C2S_CHAT	채팅
C2S_ATTACK	4 방향 근접 공격
C2S_USE_SKILL	스킬 사용(1=AoE, 2=Line, 3=Heal)
C2S_PARTY_INVITE/ACCEPT/REJECT/LEAVE	파티 풀 사이클

C2S_PICKUP/USE_ITEM/EQUIP_ITEM/UNEQUIP_ITEM/DROP_ITEM	아이템
C2S_QUEST_INTERACT/QUEST_ACTION	퀘스트 대화·수락·보상
C2S_LOGOUT	로그아웃

■ 서버 → 클라이언트 (S2C)

패킷	용도
S2C_LOGIN_RESULT / S2C_AVATAR_INFO	로그인 결과 / 본인 아바타 정보
S2C_ADD/REMOVE/MOVE_OBJECT	시야 내 객체 출현/소멸/이동
S2C_STATUS_CHANGE / S2C_MP_CHANGE	HP·EXP·레벨 / MP 갱신
S2C_ATTACK_ANIM / S2C_DAMAGE	공격 모션 / 피격 통지
S2C_DEATH / S2C_RESPAWN / S2C_LEVEL_UP	사망 / 부활 / 레벨업
S2C_SKILL_EFFECT	스킬 이펙트(시전자·방향·좌표)
S2C_PARTY_INVITED / S2C_PARTY_UPDATE	파티 초대 알림 / 파티원 상태
S2C_ITEM_DROP / S2C_ITEM_REMOVE / S2C_INVENTORY	바닥 아이템 / 인벤토리 스냅샷
S2C_QUEST_DIALOGUE / S2C_QUEST_UPDATE	대화창 지시 / 퀘스트 진행 동기화

4.3 보안 설계 — 클라이언트 좌표 불신

서버는 클라이언트가 보낸 좌표를 신뢰하지 않는다. 이동 패킷은 ① 월드 경계 클램프, ② 장애물 칸 거부, ③ 이동량(Chebyshev distance ≤ 1) 검증, ④ 이동 쿨타임(0.5 초) 검증을 모두 통과해야 반영된다. 퀘스트 상호작용도 클라가 보낸 NPC 인덱스가 아닌 서버의 플레이어 좌표 기준 근접성(장로 좌표 ± 3)을 재검증한다.

5. 자료구조

5.1 엔티티 모델 (상속 계층)

Entity (공통: id, x, y, view_list, view_lock, last_move_ms, direction)


```
└ Player (socket, RingBuffer, send_queue, hp/mp/exp/level, inventory, quests,
party_id ...)
└ NPC (type, move_mode, FSM state, spawn 좌표, target_id, active,
boss_tick_count ...)
```

- Entity::view_list (unordered_set<int>): 해당 엔티티의 시야에 들어와 있는 다른 엔티티 ID 집합. 동시 변경 시 데드락 방지를 위해 항상 (min id, max id) 순서로 락을 잡는다.
- Player 의 전투 스탯(hp/max_hp/mp/exp/level)은 모두 std::atomic 으로 보호하여 여러 워커가 동시에 접근해도 안전하다. HP 차감은 compare_exchange 루프로 "막타" 판정을 정확히 1 회만 수행한다.
- NPC 는 200,000 개를 std::unique_ptr<NPC[]> raw 배열로 한 번에 할당한다(shared_ptr 200K 의 메모리·refcount 오버헤드 회피). NPC ID = NPC_ID_START(1,000,000) + index 이므로 IsNpcId(id)로 플레이어/NPC 를 O(1)로 구별한다.

5.2 클라이언트 맵 — TBB concurrent_hash_map

```
using ClientMap = tbb::concurrent_hash_map<int, std::shared_ptr<Player>>;
ClientMap g_clients;
```

- 버킷 단위 락 + accessor 패턴으로 find/insert/erase 가 모두 동시 안전하다. 매 패킷마다 일어나는 g_clients.find 이 전역 단일 락을 거치지 않으므로 고동접 시 락 경쟁이 분산된다.
- 이름→ID 역인덱스(g_name_index, unordered_multimap)는 파티 초대 등 이름 탐색용으로 별도 유지한다.

5.3 공간 분할 — 섹터(Sector)

상수	값
월드 크기	2000 × 2000
섹터 크기	20 × 20 타일
섹터 수	100 × 100 = 10,000 개
게임 시야	15 × 15 (GAME_VIEW_RANGE = 15)
클라 표시 창	20 × 20

```
struct Sector { mutex m_lock; unordered_set<int> players; unordered_set<int> npcs; };
Sector g_sectors[100][100];
```

각 섹터는 자기 영역의 플레이어/NPC ID 집합과 전용 락을 갖는다. 시야 계산은 자기 섹터 + 인접 8 섹터(총 9 섹터)만 검사하면 되므로 전체 월드 순회가 필요 없다(AOI 최적화의 핵심).

5.4 패킷 조립 — 자체 RingBuffer

세션마다 RingBuffer(8192)를 두어 수신 바이트를 원형 큐로 적재한다. Write/Peek/Read API 로 경계를 넘는 데이터도 두 청크로 안전하게 복사하며, 패킷 단위로 정확히 추출해 process_packet 에 넘긴다. 실습의 고정 헤더 방식이 아닌 가변 길이(Length-Prefix) 조립을 자체 구현했다.

5.5 Object Pool (free-list)

OVERLAPPED_EX(송신), TimerOverlapped, DbOverlapped 를 매 이벤트마다 new/delete 하는 비용을 제거하기 위해 락 기반 free-list 풀(ObjectPool<T>)을 사용한다. 송신 풀은 8 개 샤드로 분산(g_send_pools[8])하여 락 경합을 줄이고, 객체에 pool_shard 를 박아 두어 다른 워커에서 완료되어도 원래 샤드로 정확히 반환한다.

5.6 타이머 — 샤딩 Min-Heap 우선순위 큐

```
class TimerManager {
    static constexpr int kShardCount = 8;
    struct Shard { priority_queue<TimerEvent> queue; mutex mu; condition_variable cv;
thread thread; };
    Shard shards_[8];
};
```

- TimerEvent 는 operator<를 반전시켜 priority_queue 를 Min-Heap 으로 사용한다(가장 빠른 만기가 top).
- entity_id % 8 로 8 개 샤드에 분산하여 각 샤드가 독립적인 디스패처 스레드를 갖는다. 같은 엔티티는 항상 같은 샤드로 가므로 엔티티별 처리 순서가 보존된다.
- 만기 시 콜백은 IOCP 에 IO_TIMER 이벤트를 post 할 뿐이며, 실제 처리는 워커가 담당한다.
- 타이머 이벤트 종류: NpcMove, NpcRespawn, HpRegen, MpRegen, AttackCooldown, SkillCooldown, PlayerAutoSave, GroundItemExpire.

자체 검증: --test-timer 모드는 1,000 개 더미 이벤트를 50~5,000ms 무작위 딜레이로 스케줄하고 만기 시점의 오차(lag)를 측정한다. 100ms 허용오차를 초과하는 이벤트가 0 개면 PASS.

6. 게임 흐름

6.1 접속 → 로그인 흐름 (비동기 DB Load)

```
[클라] connect
|
[서버] IO_ACCEPT → 세션 생성 + IOCP 등록 → S2C_LOGIN_RESULT(연결 성공) → WSAREcv 예약
|
[클라] C2S_LOGIN(username)
|
[서버] HandleLogin → 이름 등록 → DbWorker.EnqueueLoad(client_id, username) ※ 논블로킹
|
[DB 스레드] 백엔드 조회 → PostQueuedCompletionStatus(IO_DB_DONE)
|
[워커] OnDbResponse → OnPlayerSpawn
    ├── 기존 캐릭터: 마지막 위치·HP·EXP·레벨·인벤·장비·퀘스트 복원
    ├── 신규 캐릭터: 마을(Aetheria Village) 안 walkable 좌표에 스폰
    ├── S2C_AVATAR_INFO + S2C_INVENTORY + 퀘스트 동기화
    ├── 주변 9 섹터에서 상호 시야 구축 (S2C_ADD_OBJECT)
    └── HpRegen(5 초) · MpRegen(1 초) 타이머 시작
```

LOGIN 을 비동기화하여 DB 조회가 IOCP 워커를 블로킹하지 않는다. ID 가 DB 에 없으면 신규 캐릭터를 자동 생성하고, 동일 ID 중복 로그인은 이름 인덱스로 감지 가능한 구조를 갖춘다.

6.2 이동 + 시야 갱신 (view_list diff)

이동 패킷 수신 시 검증을 통과하면:

1. 섹터 갱신(UpdateObjectSector) 후 새 좌표 반영
2. 새 위치 주변 9 섹터에서 후보 플레이어 수집 → 시야(15×15) 필터 → new_view 계산
3. 기존 view_list 와 diff: entered(신규 진입), left(이탈), stayed(유지)로 분류
4. entered → 상호 view_list 등록 + S2C_ADD_OBJECT 양방향 전송
5. left → 상호 해제 + S2C_REMOVE_OBJECT
6. stayed → S2C_MOVE_OBJECT
7. 본인에게도 이동 확인 패킷 전송 후 SyncPlayerNpcView 로 NPC 시야 별도 동기화

플레이어 diff 와 NPC diff 의 경로를 분리하여(NPC 는 SyncPlayerNpcView 전담), 멀리 텔레포트해도 원거리 객체에 Remove 를 누락하지 않도록 했다.

6.3 사망 → 부활 흐름

플레이어 HP 가 0 이 되면 PlayerOnDeath 가:

8. S2C_DEATH 송신(자신 + 시야 내 플레이어) → 죽은 위치에 soul 이펙트
 9. 기존 시야 정리(양방향 view_list 비우기 + Remove)
 10. 사망 페널티: EXP 50% 손실, HP 풀회복(레벨은 유지)
 11. 시작 마을로 텔레포트(PLAYER_SPAWN_X/Y)
 12. 새 위치에서 시야 재구축 + S2C_RESPAWN(pillar 이펙트) + S2C_STATUS_CHANGE(HUD 갱신)
- 이는 PDF 명세 "HP 0 → 경험치 절반 깎고 HP 회복하여 시작 위치로"와 정확히 일치한다.

6.4 상태 저장 (DB Save)

- 자동 저장: 30 초 주기 PlayerAutoSave 타이머가 접속 중인 모든 플레이어 스냅샷을 EnqueueSave.
- 접속 종료 시: disconnect 경로에서 최종 상태를 강제 저장(이름이 있는, 즉 로그인 완료 세션만).

7. 핵심 알고리즘

7.1 섹터 기반 관심영역(AOI) 갱신

모든 시야 계산은 대상 좌표가 속한 섹터 + 인접 8 섹터만 순회한다. 섹터 크기(20)가 게임 시야(15)보다 크므로, 한 객체의 시야에 들어올 수 있는 모든 객체는 반드시 인접 9 섹터 안에 존재한다. 따라서 O(전체 객체)가 아닌 O(주변 섹터 내 객체)로 시야를 갱신한다.

송신은 "락 안에서는 세션 포인터만 수집, 락 밖에서 do_send 호출"하는 Send-Lockless 패턴으로 섹터 락 보유 시간을 최소화한다.

7.2 NPC AI — Lazy AI + Lua 위임

핵심 성능 전략은 Lazy AI 다.

- NPC 는 평소 비활성(active = false) 상태로 타이머에 등록되지 않는다.
- 플레이어가 NPC 시야에 진입하면(SyncPlayerNpcView) CAS(compare_exchange_strong)로 active 를 켜고 첫 NpcMove 타이머를 등록한다.
- NPC 가 매 틱(0.5 초) 처리 후 자기 view_list 가 비었으면 스스로 비활성화하고 타이머 체인을 끊는다.

→ 결과적으로 활성 AI 수는 "플레이어 근처 NPC 수"에 비례하며, 200,000 마리 전체를 항상 시뮬레이션하지 않는다. 이것이 고동접에서 AI 부하를 선형으로 억제하는 핵심이다.

AI 의사결정 자체는 Lua 의 OnTick(ctx) / OnBossTick(ctx)에 위임한다. C++는 ① view_list 에서 가장 가까운 플레이어(Chebyshev) 계산 → ② ctx 구성 → ③ Lua 호출 → ④ 반환된 이동량·타겟을 받아 월드 상태(섹터/시야/패킷)만 갱신한다.

```
NpcOnMove(npc_id):
    viewer 스냅샷 → nearest player 계산
    → GetWorkerLua().NpcTick(ctx)    # 워커 로컬 VM (락 없음)
    → target/state 갱신
    → 인접 타겟이면 공격(쿨타임 1 초, S2C_ATTACK_ANIM + S2C_DAMAGE)
    → 이동 적용 (막히면 A* 우회) → 섹터/시야 diff → Move 패킷
    → view 비었으면 비활성, 아니면 0.5 초 재스케줄
```

7.3 A* 길찾기 (장애물 우회)

Lua 가 반환한 1 칸 이동이 장애물에 막혔을 때에만 AStar::AStarStep 을 호출하여 호출 빈도를 억제한다.

- 4 방향(상하좌우) 격자 탐색, 휴리스틱은 맨해튼 거리.
- 검색 영역을 (start, goal) 바운딩 박스 + 마진 8 로 한정하고, 처리 노드 상한 256 개로 제한하여 200K NPC 가 동시에 막혀도 폭주하지 않게 했다.
- 최단 경로의 첫 한 칸 방향만 반환(매 틱 한 칸씩 이동하므로 전체 경로를 보관할 필요 없음).
- 추격 NPC 는 타겟 방향, 로밍 NPC 는 스폰 방향으로 우회한다.

7.4 데미지 적용과 막타 판정

DamageNpcAndReport 는 per-NPC view_lock 으로 데미지 적용을 직렬화하여, HP 를 0 으로 만든 호출만 사망 처리(시야 정리·섹터 제거·S2C_DEATH·리스폰 예약·EXP·드롭·퀘스트 카운트)를 정확히 1 회 실행한다. 이중 EXP/드롭/리스폰을 원천 차단하는 설계다.

7.5 세션별 송신 코일레싱 (Phase 2A 최적화)

초기에는 S2C 패킷마다 WSASend 를 호출했는데, 고동접에서 이것이 병목이었다. 이를 다음과 같이 개선했다.

- 각 세션에 send_queue(바이트 버퍼) + send_in_flight(in-flight 1 건 게이트)를 둔다.
- do_send 는 즉시 송신하지 않고 큐에 적재만 하며, idle 이면 한 번 flush_locked 를 호출한다.
- flush_locked 는 큐 앞에서 최대 8,192 바이트를 1 회 WSASend 로 합쳐 보낸다(한 틱에 쌓인 여러 패킷을 묶음 전송).
- IO_SEND 완료가 부분전송 잔여를 큐 앞에 되돌리고 다음 배치를 flush 한다(세션별 FIFO 순서 보존).
- 송신 큐가 256KB 를 넘으면 "느린 소비자"로 보고 연결을 종료한다(백프레서).

8. 게임 콘텐츠 구현

8.1 월드 — 4 대 바이옴과 장애물

월드는 4 개 구역으로 나뉘며 각 구역에 레벨대가 다른 몬스터가 배치된다.

바이옴	영역	레벨대	대표 몬스터
SW 초원	(0..960, 1040..2000)	Lv 1~10	Slime, Rat, Goblin, Wolf
SE 숲	(1040..2000, 1040..2000)	Lv 10~20	Spriggan, ForestDrake, Dryad, Centaur
NE 사막	(1040..2000, 0..960)	Lv 20~30	Mummy, Naga, Anubis, Sphinx
NW 설원	(0..960, 0..960)	Lv 30~40	IceBeast, FrostGiant, Wraith, IceDragon
중앙 마을	(960..1040)	안전지대	Aetheria Village

장애물은 Map 비트맵(2000×2000 비트 = 500KB)으로 관리한다. obstacles.txt 의 rect 스크립트로 로드한 정적 장애물에 더해, 좌표 해시 기반 절차적 장식 타일(IsProceduralDeco, 약 4.7%)을 클라/서버가 동일 해시로 생성하여 추가 데이터 전송 없이 일관된 장애물을 만든다. IsBlocked(x,y)는 비트 1 회 조회 + 해시 1 회로 $O(1)$ 이다.

8.2 캐릭터 성장

규칙	구현
레벨업 임계 EXP	$100 \times \text{level}^2$ (1→2: 100, 2→3: 400... 누진식)
레벨업 보너스	max_hp +20, HP 풀회복
HP 회복	5 초마다 max_hp 의 10% (HpRegen 타이머)
MP	최대 100, 1 초마다 +30 재생 (MpRegen 타이머)
사망 패널티	EXP 50% 손실, HP 풀회복, 시작 마을 부활

8.3 전투 시스템

- A 키 4 방향 공격: 자기 타일 + 상하좌우 4 칸의 NPC 를 동시 공격(총 5 칸 판정).
- 공격 속도: 1 초 1 회(서버 쿨타임 검증).
- 데미지: 플레이어 = $\text{level} \times 10$ + 무기 보너스, NPC = $\text{level} \times 2$.

- 전투 메시지/이펙트: S2C_ATTACK_ANIM(공격 모션) + S2C_DAMAGE(피격, 새 HP)를 시야 내 플레이어에게 브로드캐스트.
- EXP 보상: $\text{level}^2 \times 2$, Agro $\times 2$, Roaming $\times 2$, 보스는 추가 $\times 20$.

8.4 스킬 + MP 시스템

스킬	효과	쿨타임	MP
1. AoE	시전자 중심 5x5 내 모든 NPC 에 level $\times 15$	3 초	5
2. Line	바라보는 방향 직선 5 칸의 NPC 에 level $\times 20$	3 초	5
3. Heal	자신 HP 를 max_hp 의 30% 회복	10 초	10

스킬 사용 시 MP 검증 → 쿨타임 검증 → MP 차감 → S2C_SKILL_EFFECT 브로드캐스트 → 데미지/회복 적용 순으로 처리한다. 범위·방향성 공격 + 자버프성 회복을 갖춰 스킬 가산점 항목에 대응한다.

8.5 파티 시스템

- 풀 사이클: 초대(INVITE) → 수신 알림(INVITED) → 수락/거절(ACCEPT/REJECT) → 탈퇴(LEAVE), 최대 4 인.
- 경험치 공유: NPC 처치 시 온라인 파티원 수로 균등 분배(GiveExpToKillerAndParty).
- 퀘스트 처치 공유: 파티원 전원에게 처치 크레딧 적용.
- 상태 동기화: 레벨업·HP 변동 시 파티원에게 S2C_STATUS_CHANGE 전송(파티 HP 바 갱신).
- 리더 탈퇴/인원 부족 시 자동 해산 및 펜딩 초대 정리.

8.6 아이템 · 인벤토리 · 장비

items.txt 카탈로그로 3 종 아이템을 정의한다.

종류	예시	효과
소모품	HealthPotion(+50), GreatHealthPotion(+150)	즉시 HP 회복, 스택(최대 99)
무기	RustySword/IronSword/MythrilBlade	ATK +5/+15/+30
방어구	LeatherArmor/ChainMail/PlateArmor	MaxHP +30/+80/+150

- 인벤토리 20 슬롯, 소모품 스택 누적.
- 드롭/줍기/버리기: NPC 처치 시 가중치 기반 드롭 → 바닥 아이템(60 초 후 자동 소멸) → G 키 줍기(범위 1) → 버리기.

- 장착/해제: 무기는 atk_bonus 에 즉시 반영, 방어구는 max_hp 에 보너스 합산(중복 합산 방지를 위해 DB 에는 base max_hp 만 저장).
- 장착품 교체 시 기존 장비를 인벤토리로 되돌려 슬롯 오버플로가 발생하지 않도록 설계.

8.7 퀘스트 시스템 (연쇄 스토리)

quests.txt 로 정의한 3 단 연쇄 퀘스트(선행 조건 chain)를 마을 장로(NPC index 0)가 제공한다.

ID	선행	목표	보상
1	—	Slime 3 마리 처치	EXP 50 + HealthPotion ×2
2	1	Rat 5 마리 처치	EXP 120 + HealthPotion ×3
3	2	Goblin 5 마리 처치	EXP 300 + RustySword ×1

- 장로와 대화(QUEST_INTERACT) 시 서버가 보유 퀘스트 상태를 검사해 Offer/InProgress/ReadyTurnIn 중 적절한 대화창(S2C_QUEST_DIALOGUE)을 지시한다.
- 처치 시 종족 접두사 매칭으로 진행도를 카운트하고 S2C_QUEST_UPDATE 로 동기화한다.
- 퀘스트 진행/완료 상태는 DB 에 영속되어 재접속 시 복원된다.

8.8 보스 패턴 AI

4 대 바이옴마다 보스 1 마리(총 4 마리)를 배치한다.

- 스탯: Lv 50, HP 3,000, 5 분 리스폰(일반 NPC 는 30 초).
- 2 페이지 분노: HP 50% 이하 진입 시 분노 선언 채팅 + 광폭화 대사 교대 출력.
- 광역기(AoE): 5 틱마다 반경 3 칸 내 모든 플레이어에 광역 데미지.
- 도발 채팅: 전투 시작:분노 시 S2C_CHAT_MESSAGE 로 보스 대사 출력("You dare enter my domain?!" 등).
- 확장 어그로: 일반 NPC(범위 5)보다 넓은 범위 10 으로 플레이어를 인식.

보스 AI 는 npc_ai.lua 의 OnBossTick 이 페이즈·채팅·AoE 발동 여부를 결정하고 C++가 실제 데미지/브로드캐스트를 수행한다.

9. 데이터베이스 연동

9.1 백엔드 추상화 + 자동 폴백

IDbBackend (인터페이스: Load / Save)

└─ MySQLdbBackend (ODBC, 정규화 3 테이블) ← db.ini enabled=1 & 연결성공
└─ JsonFileBackend (data/players/*.json) ← 폴백

data/db.ini 의 enabled/conn 을 읽어 MySQL(ODBC) 연결을 시도하고, 연결 실패 시 자동으로 JSON 파일 백엔드로 폴백한다. 따라서 DB 드라이버가 없는 머신에서도 서버가 항상 동작한다(무크래시 검증됨).

9.2 비동기 처리 흐름

[워커] EnqueueLoad/Save → [DbWorker 단일 스레드] 큐 폴링 → 백엔드 동기 호출
→ PostQueuedCompletionStatus(IO_DB_DONE) → [워커] OnDbResponse

게임 스레드(워커)는 Job 큐에 push 만 하고 즉시 반환하며, 전용 DB 스레드가 직렬로 처리한 뒤 IOCP 로 결과를 돌려준다. DB 지연이 게임 루프를 절대 블로킹하지 않는다.

9.3 정규화 스키마

PlayerSnapshot(username PK + 스칼라 + inventory[] + quests[])을 3 개 테이블로 정규화 저장한다.

테이블	주요 컬럼
players	username(PK), hp, max_hp(base), exp, level, x, y, equipped_weapon_id, equipped_armor_id
player_inventory	username(FK), slot, item_id, qty
player_quests	username(FK), quest_id, kill_count, state

FK 는 ON DELETE CASCADE, 계정은 ODBC 호환을 위해 mysql_native_password 를 사용한다.

로그인 1 회로 스칼라·장비·인벤·퀘스트·base max_hp 보정까지 전부 라운드트립 복원됨을 검증했다.

기획서에서는 MS-SQL + ADO 를 계획했으나, 실제 구현에서는 MySQL + ODBC 로 연동하고 JSON 폴백을 추가하여 이식성과 안정성을 높였다.

10. 스크립트(Lua) 연동

10.1 워커 로컬 VM (성능 핵심)

lua_State 는 thread-safe 가 아니므로, 초기에는 단일 전역 VM + 단일 mutex 로 모든 AI 틱을 직렬화했는데 이것이 약 3,000 CCU 에서 병목이었다. 이를 워커 스레드마다 thread_local lua_State 를 두는 방식으로 개선했다.

- OnTick/OnBossTick 은 모든 상태를 ctx 로 전달받는 stateless 함수이므로 어느 VM 에서 실행해도 결과가 동일하다.
- 따라서 AI 핫패스(NpcTick/BossTick)는 락을 전혀 잡지 않으며, 인터프리터 개수는 NPC 수가 아닌 워커(코어) 수에 비례한다.
- 워커별 고유 WORKER_SEED 를 주입하여 모든 워커가 동일한 로밍 난수 시퀀스를 도는 것을 방지한다.

10.2 스크립트로 정의하는 게임 데이터

스크립트	역할
npc_spawn.txt	몬스터 종류·레벨·HP·타입·이동모드·배치 영역 (200K 배치)
npc_ai.lua	NPC AI(OnTick) + 보스 AI(OnBossTick)
obstacles.txt	장애물 rect 배치
items.txt / quests.txt	아이템·퀘스트 카탈로그

PDF 의 추가 요소 "배치 좌표·이름·TYPE·Level 등 모든 정보를 Script 로 저장"과 "보스 몬스터의 다양한 공격 패턴(이동/채팅/스킬공격)"을 스크립트로 구현하여 스크립트 가산점에 대응한다. 스폰은 고정 시드를 써서 결정적 배치(동일 스크립트 → 동일 월드)를 보장한다.

11. 성능 최적화 및 동접 측정

11.1 적용한 최적화 종합

영역	최적화	효과
시야	섹터 기반 AOI(9 섹터만 검사)	전체 순회 제거
NPC AI	Lazy AI(플레이어 근처만 활성화)	활성 AI = 동접 비례
스크립트	워커 로컬 Lua VM	전역 Lua 락 직렬화 제거
타이머	8 샤드 Min-Heap	단일 타이머 스레드 병목 제거
송신	8 샤드 송신 풀 + 세션 코알레싱	패킷당 WSASend 폭주 제거
네트워크	TCP_NODELAY(Nagle off)	원거리 RTT 링크의 패킷당 Nagle 지연 제거
메모리	Object Pool, NPC raw 배열	new/delete 제거, 풋프린트 최소화

클라이언트 맵	TBB concurrent_hash_map	버킷 단위 락으로 경합 분산
빌드	Release /O2	—

11.2 병목 진단 및 개선 단계

초기 부하 분석에서 NPC AI 부하가 $O(\text{동접} \times \text{활성 NPC} \times 2 \text{ 틱/초})$ 로 증가하는데 ① 단일 Lua 락 ② 단일 타이머 스레드 ③ 전역 송신 풀 락의 직렬 구간을 통과해 약 3,000 CCU 에서 한계에 부딪혔다. Script(Lua)는 PDF 필수 스펙이라 제거할 수 없으므로 유지하되 병렬화하는 방향으로 다음을 적용했다.

- Phase 1: 워커 로컬 Lua VM, 8 샤드 타이머, 8 샤드 송신 풀 → 3,000 벙 제거.
- Phase 2A: 세션별 송신 코일레싱(패킷당 WSASend → 세션 큐 + in-flight 1 건 게이트 + ≤8KB 1 회 WSASend), 송신 완료 시 풀 객체 회수(누수 수정).
- 분리 머신 측정 대비(네트워크 지연 제거): 서버 accept 소켓과 부하 클라이언트 소켓에 TCP_NODELAY 를 설정해, 원거리/VPN RTT 링크에서 Nagle+delayed-ACK 가 작은 이동 패킷을 묶어 패킷당 수십~수백 ms 지연을 유발하는 것을 차단했다(localhost RTT~0 에선 무영향). 아울러 부하 클라이언트의 진단 출력을 전 스레드 합산 초당 1 회로 throttle 해, 5,000 CCU 부근 접속 실패 폭주 시 동기 콘솔 flush 가 측정 스레드를 막아 지연 수치가 부풀려지던 문제를 제거했다.

11.3 측정 결과

- 동일 머신(localhost)에서 서버 + 200K NPC + 헤드리스 부하 클라이언트를 함께 구동하여 측정.
- 피크 **3,462 CCU 에서 처리 지연 0~1ms** 도달, 3,000 CCU 벙 해소.
- 이 시점의 한계는 서버 처리 능력이 아니라 단일 머신에서 클라이언트와 서버가 CPU 를 경합하는 환경 제약이었다(분리 머신 측정 시 추가 여유 예상).

측정 방법: PC 한 대에 서버를 띄우고 다른 PC(또는 동일 PC)에서 테스트 클라이언트를 실행하는 PDF 의 동접 판단 기준에 따른다. GUI 스트레스 클라이언트와 stdin 으로 IP 를 받는 헤드리스 클라이언트 두 가지를 검증 자산으로 준비했다.

12. 채점 기준 대응 정리

평가 항목	배점	대응 구현
구현	50%	이동·충돌, 장애물(비트맵+절차적 장식), Peace/Agro NPC AI, NPC 이동(로밍/고정/A* 우회), 4 방향 전투, MySQL DB 영속화, Lua

		스크립트, 채팅, 전투 메시지 — 전부 구현
성능	20%	IOCP + 섹터 AOI + Lazy AI + 샤딩 타이머/송신 + 코알레싱 → localhost 3,462 CCU @ 0~1ms
설명서	10%	본 보고서(프로토콜·자료구조·게임 흐름·알고리즘·DB) + 기획서
게임성	40%	4 대 바이옴, 스킬 3 종+MP, 파티(경험치 공유), 아이템·인벤·장비, 연쇄 퀘스트, 보스 2 페이지 패턴
스크립트 가산점	+α	Lua 몬스터 배치 + 보스 패턴 AI(이동/채팅/광역기)
파티 가산점	+10	초대/수락/거절/탈퇴 풀 사이클 + 경험치 공유
아이템 가산점	+α	소모품 사용·누적/스택·인벤토리·장비 장착/해제
퀘스트 가산점	+α	사냥 퀘스트·진행상태 표시·NPC 대화·연쇄 퀘스트
스킬 가산점	+α	범위(AoE)·방향성(Line) 공격 + 회복

13. 한계 및 향후 과제

- 분리 머신 5,000 CCU 검증 미완: localhost 측정에서 3,462 CCU(0~1ms)를 달성했으나, 서버/클라이언트를 물리적으로 분리한 환경에서 5,000 CCU·지연 <100ms 정식 측정은 추가 과제로 남아 있다.

기획 대비 실제 구현 차이 (현실적 일정 조정 결과):

- DB: MS-SQL/ADO 계획 → MySQL/ODBC + JSON 풀백으로 구현(이식성·안정성 우선).
- 스크립트: LuaJIT + sol2 계획 → Lua 5.5 C API 직접 바인딩(워커 로컬 VM 병렬화에 집중).
- 길찾기: JPS 계획 → 격자 A*(노드 상한 256)로 구현(빈도 제어로 충분한 성능 확보).
- 스킬/퀘스트: 스킬트리·12 단 퀘스트 계획 → 스킬 3 종 + 3 단 연쇄 퀘스트로 핵심 게임성 우선 구현.
- 추가 병렬화 여지: NPC 틱 배칭/LOD, 섹터 단위 Read-Write 락 등 Phase 2B/2C 최적화로 추가 동접 확보 가능.

부록 A. 주요 상수 (GameConfig.h)

상수	값	의미
NPC_TICK_INTERVAL_MS	500	NPC·플레이어 이동 주기(0.5 초/칸)
AGRO_DETECT_RANGE	5	Agro 감지 반경(11×11)
ROAM_AREA_RANGE	10	로밍 반경(20×20)
NPC_RESPAWN_MS	30,000	NPC 부활(30 초)
HP_REGEN_INTERVAL_MS / PERCENT	5,000 / 10	HP 5 초마다 10% 회복
ATTACK_INTERVAL_MS	1,000	공격 쿨타임(1 초)
BASE_DAMAGE_PER_LEVEL	10	플레이어 데미지 = level×10
BOSS_MAX_HP / BOSS_RESPAWN_MS	3,000 / 300,000	보스 HP / 5 분 리스폰
NUM_NPCS	200,000	총 NPC 수
SECTOR_SIZE	20	섹터 한 번(100×100 섹터)
GAME_VIEW_RANGE	15	게임 시야(15×15)

부록 B. 빌드 및 실행

- DB(선택): mysql -u root -p < data/schema_mysql.sql 로 스키마 생성, data/db.ini 에서 enabled 와 conn(ODBC 연결 문자열) 설정. (미설정 시 JSON 폴백)
- 서버: Visual Studio 에서 MMOSERVER_Termproject 를 x64 Release 로 빌드. 실행 시 프로젝트 루트에서 실행하여 data/ 스크립트를 찾도록 한다.
- 클라이언트: CLIENT/client_sample(SFML) 빌드 후 서버 IP 로 접속.
- 타이머 자체 검증: 서버 실행 파일에 --test-timer 인자를 주면 1,000 개 이벤트의 만기 정확도 PASS/FAIL 을 출력한다.

— 보고서 끝 —